

Isotope spans for OpenTelemetry (OTel)

Optional span writer for `kafka-isotope`. It turns every isotope hop into an **OpenTelemetry span** and writes it to a Kafka topic as an OTLP `ExportTraceServiceRequest` — the exact bytes a stock [OpenTelemetry Collector](#) reads.

Nothing isotope-specific runs on the consuming side. A Collector with a `kafka` receiver set to `encoding: otlp_proto` consumes the topic and exports to any OTLP backend (Groundcover, Jaeger, Tempo, Honeycomb...). The integration a customer has to perform is Collector config plus read access to one topic.



Table of Contents

- [1.0 Install](#)
- [2.0 Put It To Work](#)
 - [2.1 Collector Config](#)
- [3.0 What a Span Looks Like](#)
 - [3.1 Where Produce Spans Come From](#)
- [4.0 What It Costs a Send](#)
 - [4.1 It Has Its Own Producer](#)
 - [4.2 Lifecycle](#)
- [5.0 Known gaps](#)

1.0 Install

```
dependencies {
    implementation 'ai.signalroom:kafka-isotope-core:0.19.0'
    implementation 'ai.signalroom:kafka-isotope-otel:0.19.0' // optional -
    only for spans
}
```

2.0 Put It To Work

Start the writer once at boot, after which the interceptor you already registered starts producing spans. Nothing else in your code changes.

```
import ai.signalroom.kafka.isotope.otel.KafkaOtlpSpanSink;

Map<String, Object> spanProducerConfig = Map.of(
    ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "kafka:9092");

KafkaOtlpSpanSink.start(spanProducerConfig);           //
isotope_trace_spans
KafkaOtlpSpanSink.start(spanProducerConfig, "my_spans_topic"); // or
name your own
```

At shutdown, `KafkaOtlpSpanSink.close()` hands back the reference `start` took. The span producer itself goes away once every interceptor has closed too (see §4.2 Lifecycle).

If the producer can't be built — bad `bootstrap.servers`, a security misconfiguration — the failure is logged at WARN and swallowed. Nothing is registered, spans stay off, and your pipeline runs on. Same posture as the Prometheus exporter: an optional sidecar never takes the application down with it.

2.1 Collector Config

```
receivers:
  kafka:
    brokers: [kafka:9092]
    traces:
      topics: [isotope_trace_spans]
      encoding: otel_proto
      group_id: otel-collector-isotope

exporters:
  otlp:
    endpoint: <your-otlp-endpoint>:4317

service:
  pipelines:
    traces:
      receivers: [kafka]
      exporters: [otlp]
```

On Collector versions before the per-signal split, `topic` and `encoding` sit at the top level of the receiver instead of under `traces:`. The payload is the same either way.

3.0 What a Span Looks Like

OTel field	From
<code>trace_id</code>	the isotope trace id, verbatim — a 16-byte UUIDv7 is a 128-bit OTel trace id

OTel field	From
<code>span_id</code>	derived: <code>sha256(trace_id, service, topic, ts)[0..8]</code>
<code>parent_span_id</code>	the same derivation over the previous entry in the hop list
<code>name / kind</code>	<code>send <topic></code> / PRODUCER, or <code>receive <topic></code> / CONSUMER
<code>start → end</code>	previous hop's timestamp → this hop's timestamp
<code>service.name</code> (resource)	the service on this edge
<code>status</code>	ERROR, with the failure message, when the produce failed; otherwise unset
attributes	<code>messaging.system</code> , <code>messaging.destination.name</code> , <code>messaging.operation.name</code> , <code>messaging.operation.type</code> , <code>isotope.pipeline</code> , <code>isotope.origin_service</code> , <code>isotope.hop_count</code> , <code>isotope.truncated</code>
delivery attributes	<code>messaging.destination.partition.id</code> and <code>messaging.kafka.offset</code> when the record was written, <code>error.type</code> when it wasn't. Produce spans on kafka-clients 4.1+ only (see §3.1)

Derived span ids are the trick that makes this work without a wire-format change. Hops carry no span id and never will — adding one would break every deployed reader. Instead each span's id is a hash of what the header already carries, so a service computes its parent's id from the previous entry in `h[]` and lands on exactly the id that parent computed for itself. Spans link into a tree across services with no shared state and nothing new on the wire.

The corollary: the hash is a compatibility surface. Change its inputs or encoding and spans from an older service stop matching parent ids computed by a newer one.

Durations are real, not zero. A hop is a single timestamp, so a span runs from the *previous* hop's timestamp to this one's, making each span the edge between two services rather than a point. Those two timestamps come from different machines, so a backwards clock is clamped to a zero-length span rather than a negative one.

3.1 Where Produce Spans Come From

On **kafka-clients 4.1+**, a produce span is emitted once the broker has answered, from `onAcknowledgement`. Kafka 4.1 added an overload of that callback that receives the headers of the record as sent, and those still carry the `x-isotope` JSON that `onSend` wrote. Its last hop is this produce, so the acknowledgement has everything the span needs, and no map of in-flight records is required to match the answer back to its trace. The span therefore records the outcome: partition and offset when the record was written, ERROR status and `error.type` when it wasn't, including sends that fail before reaching the broker.

On **older clients**, the acknowledgement callback gets no headers, so the span is emitted from `onSend` instead and records the attempt only (see §5.0).

The interceptor detects which applies when it loads, so nothing needs configuring. Span ids are derived from the hop, not from where the span is emitted, so they are identical on both paths. A pipeline mixing client versions still links into one trace.

4.0 What It Costs a Send

Nothing that can be measured on the caller's thread or on the producer's network thread, by construction:

- On kafka-clients 4.1+, the acknowledgement runs on the producer's network I/O thread, where slow code would delay every other send on that producer. So it only reads the `x-isotope` header bytes and queues them. Decoding the JSON waits for the writer thread.
- On older clients, `onSend` copies a handful of fields into a small event and queues that instead.
- Either way, it's a non-blocking offer onto a bounded queue (10,000 by default). Hashing, JSON decoding, protobuf encoding and the Kafka write all happen on a background daemon thread.
- Queue full $\Rightarrow$ the span is dropped and counted (`Kafka0tlpSpanSink.droppedSpans()`), never queued behind a slow broker. Telemetry yields to the pipeline.
- Every error on the path is swallowed and logged, throttled to at most one warning a minute so a broker outage can't become a log flood.
- Span records are keyed by trace id, so one trace's spans stay on one partition and reach the Collector in order.

4.1 It Has Its Own Producer

The writer owns a **separate producer**, shared by every interceptor in the JVM and built with `interceptor.classes` stripped, so its writes are never themselves traced. The interceptor cannot reuse the producer it is attached to: that one is `mid-send()`, and its interceptor chain would stamp the span records and trace the tracer. The consume-marker producer is separate for the same reason — the difference is that one is yours to manage and this one is ours.

Your config is used as given, except for what would make the producer trace itself or block: `interceptor.classes` is removed, the serializers are forced to `ByteArraySerializer`, and `acks=1`, `enable.idempotence=false`, `linger.ms=50`, `max.block.ms=1000` are applied *only if you didn't set them*.

4.2 Lifecycle

`start()` takes the first reference and each configured interceptor takes one more. An interceptor's `close()` hands one back, `Kafka0tlpSpanSink.close()` hands back the starter's, and the producer shuts down when the last reference goes. This is why closing one producer can't pull the writer out from under its siblings — a real hazard for a JVM-wide resource that every interceptor shares. Shutdown drains whatever is still queued before closing.

5.0 Known gaps

- **On kafka-clients older than 4.1, produce spans record the attempt, not the outcome.** Those clients pass `onAcknowledgement` no headers, so spans are emitted from `onSend` (see §3.1). A record the producer later fails to deliver still gets an ordinary span, and no span carries a partition or offset. Upgrade to 4.1+ to get the outcome.

- **Consume spans need the full header.** `IsotopeContext.recordConsume` emits a consume span only when the record still carries the `x-isotope` JSON, since the span's parent comes from the hop list. Records carrying only the scalar headers — including the consume markers themselves — produce no span, which is what we want: markers are report input, not pipeline edges.
- **No span events or links.** There is nothing in an isotope header to populate them with. Only failed produces carry a status; consume spans and successful produces leave it unset, as OTel recommends.